Apellidos: Gómez González
Nombre: Isaac
DNI: 49968477B
Asignatura: PCD
Grupo: 2 **Subgrupo:** 3
Convocatoria: Junio
Año académico: 2025–2026
Profesor: Cánovas Rodríguez, Adrián

MEMORIA DE PRÁCTICAS

Entregables 2025–2026

Índice

1. Ejercicio 1: ReentrantLock **2**
1.1. Recursos no compartibles 2
1.2. Condiciones de sincronización 2
1.3. Pseudocódigo comentado 2
1.4. Cuestiones planteadas en los ejercicios 4
1.5. Código Java de los ejercicios comentado 5

2. Ejercicio 2: Semáforos **9**
2.1. Recursos no compartibles 9
2.2. Condiciones de sincronización 9
2.3. Pseudocódigo comentado 10
2.4. Cuestiones planteadas en los ejercicios 12
2.5. Código Java de los ejercicios comentado 13

3. Ejercicio 3: Monitores **16**
3.1. Recursos no compartibles 16
3.2. Condiciones de sincronización 16
3.3. Pseudocódigo comentado 17
3.4. Cuestiones planteadas en los ejercicios 23
3.5. Código Java de los ejercicios comentado 24

4. Ejercicio 4: Paso de Mensajes Asíncrono **30**
4.1. Recursos no compartibles 30
4.2. Condiciones de sincronización 30
4.3. Pseudocódigo comentado 31
4.4. Cuestiones planteadas en los ejercicios 34
4.5. Código Java de los ejercicios comentado 35

1. Ejercicio 1: ReentrantLock

1.1. Recursos no compartibles

Unicamente la pantalla, ya que es el recurso que ambos hilos necesitan para imprimir sus resultados.

1.2. Condiciones de sincronización

El uso de un cerrojo para bloquear la sección crítica de la pantalla. Que permite a los hilos el acceso a la pantalla en exclusión mutua, evitando así que ambos hilos impriman al mismo tiempo y mezclen sus salidas.

1.3. Pseudocódigo comentado

```
Algoritmo: Clase Main
// Variables globales
Datos:
cerrojoPantalla : Cerrojo
NUM_OPERACIONES : Entero
LONGITUD_FILAS_Y_COLUMNAS : Entero
// Inicialización de variables globales
NUM_OPERACIONES ← 10
LONGITUD_FILAS_Y_COLUMNAS ← 3
Función main()
    initial (cerrojoPantalla, 1)
    cobegin
        suma()
        multiplicar()
    coend
fin
```

```
Algoritmo: Clase SumaMatriz
// Variables de instancia
Datos:
metricasDeEntrenamiento : Entero[][]
impactoAcumulado : Entero[][]
Función suma()
    para i ← 1 a NUM_OPERACIONES hacer
        generarMatriz (metricasDeEntrenamiento)
        // Calcular suma de matrices
        // Guardar resultado en impactoAcumulado

        wait(cerrojoPantalla)
        // Impresión
        signal(cerrojoPantalla)
    fin
fin
```

```
Algoritmo: Clase CuadradoMatriz
// Variables de instancia
Datos:
metricasDeEntrenamiento : Entero[][]
impactoAcumulado : Entero[][]
Función multiplicar()
    para i ← 1 a NUM_OPERACIONES hacer
        generarMatriz (metricasDeEntrenamiento)
        // Calcular cuadrado de matrices
        // Guardar resultado en impactoAcumulado

        wait(cerrojoPantalla)
        // Impresión
        signal(cerrojoPantalla)
    fin
fin
```

1.4. Cuestiones planteadas en los ejercicios

a) **Si no usas sincronización, ¿cómo podría ser la salida por pantalla?**

Como regla general, si no usas sincronización, la salida por pantalla es caótica e impredecible. En este caso, dado que ambos hilos (SumaMatriz y CuadradoMatriz) están accediendo a la pantalla para imprimir sus resultados, sin sincronización podrían intercalarse las impresiones de ambos hilos. Esto podría resultar en una salida mezclada donde partes de la matriz de SumaMatriz se imprimen junto con partes de la matriz de CuadradoMatriz, haciendo que sea difícil o imposible entender los resultados.

En el caso particular de java cada System.out.println() es una operación atómica, lo que significa que cada llamada a esta función se ejecutará completamente sin ser interrumpida por otro hilo. Sin embargo, esto no garantiza que las impresiones de ambos hilos no se mezclen, ya que cada hilo podría imprimir varias líneas de la matriz antes de que el otro hilo tenga la oportunidad de imprimir. Por lo tanto, aunque cada línea individual se imprima de manera atómica, la secuencia completa de impresiones podría resultar en una salida caótica si no se sincroniza adecuadamente.

1.5. Código Java de los ejercicios comentado

Clase Main

```java
package ejercicio1; //NOSONAR

import java.util.Random;
import java.util.concurrent.locks.ReentrantLock;

public class Main {
    public static final int NUM_OPERACIONES = 10;
    public static final int LONGITUD_FILAS_Y_COLUMNAS = 3;
    public static final int VALOR_MAX_MATRIZ = 10;
    public static final int VALOR_MIN_MATRIZ = 1;
    public static final Random RANDOM = new Random();

    /* Este cerrojo se encarga de bloquear la sección crítica de la pantalla para que no acceda
    * más de un hilo a la vez */
    public static final ReentrantLock CERROJO_PANTALLA = new ReentrantLock();

    /**
    * Función que le da valores aleatorios entre 1 y 9 a una matriz de 3x3,
    * @param matriz Es la matriz que se rellenara con núevos valores aleatorios
    */
    public static void generarMatriz(int[][] matriz){
        for(int i = 0; i < LONGITUD_FILAS_Y_COLUMNAS; i++) {
            for(int j = 0; j < LONGITUD_FILAS_Y_COLUMNAS; j++) {
                matriz[i][j] = VALOR_MIN_MATRIZ +
                        RANDOM.nextInt(VALOR_MAX_MATRIZ);
            }
        }
    }


    /**
    * Función principal del programa donde se ejecutan dos hilos, uno que se encarga de
    * hacer el cuadrado de una matriz y otro el doble de una matriz.
    * @param args no hay nada en esta variable.
    */
    public static void main(String[] args) {
        CuadradoMatriz cuadrado = new CuadradoMatriz();
        SumaMatriz suma = new SumaMatriz();
        cuadrado.start();
        suma.start();

        try {
            cuadrado.join();
            suma.join();
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();;
        }

        System.out.println("Fin␣del␣hilo␣principal."); //NOSONAR
    }
}
```

Clase SumaMatriz

```
package ejercicio1; //NOSONAR

public class SumaMatriz extends Thread{

    private final int[][] metricasDeEntrenamiento =
            new int[Main.LONGITUD_FILAS_Y_COLUMNAS][Main.LONGITUD_FILAS_Y_COLUMNAS];

    /**
    * Función para sumar la matriz consigo misma.
    * @param matriz matriz que se va a sumar consigo misma
    * @return matriz sumada consigo misma.
    */
    private int[][] calcularSumaMatriz(int[][] matriz) {
        int[][] suma = new int[matriz.length][matriz[0].length];
        for (int i = 0; i < matriz.length; i++) {
            for (int j = 0; j < matriz[0].length; j++) {
                suma[i][j] = matriz[i][j] + matriz[i][j];
            }
        }
        return suma;
    }

    /**
    * Imprime en el formato correcto del ejercicio.
    * @param matriz es la que se usa para hacer la suma.
    * @param solucion es la solución del cálculo.
    */
    @SuppressWarnings("java:S106")
    private void imprimirSumaDeMatriz(int[][] matriz, int[][] solucion) {
        System.out.println("A␣+␣A\n");

        for (int[] fila : matriz) {
            for (int valor : fila) {
                System.out.print(valor + "a␣");
            }
            System.out.println();
        }
        System.out.println("\n+\n");

        for (int[] fila : matriz) {
            for (int valor : fila) {
                System.out.print(valor + "a␣");
            }
            System.out.println();
        }

        System.out.println("\n2A\n");

        for (int[] fila : solucion) {
            for (int valor : fila) {
                System.out.print(valor + "r␣");
            }
            System.out.println();
        }
        System.out.println();
    }

    @Override
    public void run() {

        int[][] impactoAcumulado;

        for (int operacionesRealizadas = 0; operacionesRealizadas < Main.NUM_OPERACIONES;
            operacionesRealizadas++) {

            Main.generarMatriz(metricasDeEntrenamiento);
            // Suma de la matriz A + A (2A)
            impactoAcumulado = calcularSumaMatriz(metricasDeEntrenamiento);

            /* Aquí se bloquea el cerrojo de la pantalla para asegurar que este recurso no
            * compartible no sea usado por más de un proceso a la vez
            */
            Main.CERROJO_PANTALLA.lock();

            imprimirSumaDeMatriz(metricasDeEntrenamiento, impactoAcumulado);

            Main.CERROJO_PANTALLA.unlock();

        }

    }
}
```

Clase CuadradoMatriz

```
package ejercicio1; //NOSONAR

public class CuadradoMatriz extends Thread {

    private final int[][] metricasDeEntrenamiento =
            new int[Main.LONGITUD_FILAS_Y_COLUMNAS][Main.LONGITUD_FILAS_Y_COLUMNAS];

    /**
    * Función para calcular el cuadrado de una matriz.
    * @param matriz es a la que se le va ha realizar el cuadrado.
    * @return devuelve el cuadrado de la matriz.
    */
    private int[][] calcularCuadradoMatriz(int[][] matriz) {
        int[][] cuadrado = new int[matriz.length][matriz[0].length];
        for (int i = 0; i < matriz.length; i++) {
            for (int j = 0; j < matriz.length; j++) {
                cuadrado[i][j] = 0;
                for (int k = 0; k < matriz.length; k++) {
                    cuadrado[i][j] += matriz[i][k] * matriz[k][j];
                }
            }
        }
        return cuadrado;
    }

    /**
    * Imprime en el formato correcto del ejercicio.
    * @param matriz es la que se usa para hacer el cuadrado.
    * @param solucion es la solución del cálculo.
    */
    @SuppressWarnings("java:S106")
    private void imprimirCuadradoDeMatriz(int[][] matriz, int[][] solucion ) {
        System.out.println("A␣x␣A\n");

        for (int[] fila : matriz) {
            for (int valor : fila) {
                System.out.print(valor + "a␣");
            }
            System.out.println();
        }
        System.out.println("\nx\n");

        for (int[] fila : matriz) {
            for (int valor : fila) {
                System.out.print(valor + "a␣");
            }
            System.out.println();
        }

        System.out.println("\nA2\n");

        for (int[] fila : solucion) {
            for (int valor : fila) {
                System.out.print(valor + "r␣");
            }
            System.out.println();
        }
        System.out.println();
    }
```

```
    @Override
    public void run() {

        int[][] impactoAcumulado;

        for (int operacionesRealizadas = 0; operacionesRealizadas < Main.NUM_OPERACIONES;
            operacionesRealizadas++) {

            Main.generarMatriz(metricasDeEntrenamiento);
            impactoAcumulado = calcularCuadradoMatriz(metricasDeEntrenamiento);

            /*
            * Aquí se bloquea el cerrojo de la pantalla para asegurar que este recurso no
            * compartible no sea usado por más de un proceso a la vez.
            */
            Main.CERROJO_PANTALLA.lock();

            imprimirCuadradoDeMatriz(metricasDeEntrenamiento, impactoAcumulado);

            Main.CERROJO_PANTALLA.unlock();

        }

    }
}
```

2. Ejercicio 2: Semáforos

2.1. Recursos no compartibles

Para este problema cada panel individual no es compartible, ya que cada uno se comporta como una pantalla independiente, por ello es necesario protegerlos. Para protegerlos se crea una variable **panelesLibres** que almacena un booleano para cada panel, que también es un recurso no copartible, porque podría estar siendo usa tanto por un hilo para leer como por otroç para escribir.

2.2. Condiciones de sincronización

Para garantizar las condiciones de sincronización se crean dos semáforos: **semaforoDePaneles** y **mutexVariablesCompartidas**. El primero se encarga de bloquear la sección crítica de la pantalla para que no accedan más de tres procesos a los paneles al mismo tiempo, siendo necesario que los hilos que acceden a los paneles se bloqueen en la sección crítica de la pantalla si no hay paneles libres y se liberan cuando hay alguno disponible, siendo esto una condición de sincronización. Con el segundo semáforo se comprueba que alguíen no esté usando ya el array **panelesLibres** para evitar que uno pueda escribir y otro leer a la vez, protegiendo así un recurso no compartible, hasta que quede libre.

2.3. Pseudocódigo comentado

```
Algoritmo: Clase Main
Datos:
    paneles : Panel[] // Paneles disponibles
    panelesLibres : Booleano[]
    semaforoDePaneles : Semaforo
    mutexVariablesCompartidas : Semaforo
    NUMERO_DE_PANELES : Entero
    NUMERO_DE_EQUIPOS : Entero
    NUMERO_DE_DEPORTISTAS_POR_EQUIPO : Entero
    NUMERO_DE_SECUENCIAS : Entero
    NUMERO_DE_EJERCICIOS : Entero
    equipos : Equipo[]
// Inicialización de variables globales
NUMERO_DE_PANELES ← 3
NUMERO_DE_EQUIPOS ← 10
NUMERO_DE_DEPORTISTAS_POR_EQUIPO ← 4
NUMERO_DE_SECUENCIAS ← 3
NUMERO_DE_EJERCICIOS ← 5
Función main()
    initial(semaforoDePaneles, NUMERO_DE_PANELES)
    initial(mutexVariablesCompartidas, 1)
    para i ← 1 a NUMERO_DE_PANELES hacer
        panelesLibres[i] ← true
    fin
    cobegin
        para i ← 1 a NUMERO_DE_EQUIPOS hacer
            equipos[i].equipo(i)
        fin
    coend
fin
```

```
Algoritmo: Clase Equipo
// Variables de la instancia
Datos:
tablaDeMatricesDeEjercicios : Entero[][][]
MatrizResumenR : Entero[][]
idEquipo : Entero

Función equipo(idEquipo)
    para i ← 1 a NUMERO_DE_SECUENCIAS hacer
        // Generar tablaDeMatricesDeEjercicios
        // Rellenar MatrizResumenR

        wait(semaforoDePaneles)
            wait(mutexVariablesCompartidas)
                // Buscar panel disponible
                // Poner a false el panel escogido
                // Guadar idPanel escogido
            signal(mutexVariablesCompartidas)
            // Imprimir en panel escogido
            wait(mutexVariablesCompartidas)
                panelesLibres [idPanel] = true
            signal(mutexVariablesCompartidas)
        signal(semaforoDePaneles)
    fin
fin
```

2.4. Cuestiones planteadas en los ejercicios

Según las condiciones del problema:

a) **(0.5 puntos sobre 10) Si consideramos un panel solamente ¿cómo es el acceso al mismo? Si consideramos los tres paneles ¿cómo es el acceso? Explica ambas situaciones.**
En el caso de tener un unico panel, el acceso al mismo se realiza mediante un mutex, que se encarga de bloquear la sección crítica de la pantalla para que esta no pueda ser usada por más de un proceso a la vez.

En el caso de los tres paneles, hay que garantizar que no accede más de un hilo a cada panel, pero esto permite que varios hilos impriman sus resultados en paralelo ya que es como si hubieran tres pantallas independientes, por lo que solo hay que proteger la selección de la pantalla que es mucho más rápido que esperar a que un hilo use la pantalla.

b) **(0.5 puntos sobre 10) ¿Qué tipo de semáforos has usado para resolver el problema (binarios, generales)?**
Uno binario para proteger la selección del panel y uno general para controlar el número de paneles disponibles.

2.5. Código Java de los ejercicios comentado

Clase Main

```
package ejercicio2; //NOSONAR

import java.util.Random;
import java.util.concurrent.Semaphore;

public class Main {

    public static final int NUMERO_DE_PANELES = 3;
    public static final int NUMERO_DE_EQUIPOS = 10;
    public static final int NUMERO_DE_DEPORTISTAS_POR_EQUIPO = 4;
    public static final int NUMERO_DE_SECUENCIAS = 3;
    public static final int NUMERO_DE_EJERCICIOS = 5;
    public static final int INICIALIZAR_SEMAFORO_BINARIO = 1;
    public static final int VALOR_MAX_MATRIZ = 9;
    public static final int VALOR_MIN_MATRIZ = 1;
    public static final Random RANDOM = new Random();

    public static final int NORMALIZAR_NUMERO_DE_PANEL = 1;
    public static final int NORMALIZAR_POSICION_X_DE_PANEL = 415;
    public static final int NORMALIZAR_POSICION_Y_DE_PANEL = 0;

    protected static final Panel[] PANELES = new Panel[NUMERO_DE_PANELES];
    protected static final boolean[] PANELES_LIBRES = new boolean[NUMERO_DE_PANELES];
    public static final Semaphore MUTEX_VARIABLES_COMPARTIDAS =
        new Semaphore(INICIALIZAR_SEMAFORO_BINARIO);
    public static final Semaphore SEMAFORO_DE_PANELES = new Semaphore(NUMERO_DE_PANELES);

    protected static final Equipo[] EQUIPOS = new Equipo[NUMERO_DE_EQUIPOS];

    /**
    * Función que le da valores aleatorios entre 1 y 10 a una matriz de 5x5,
    * @param matriz Es la matriz que se rellenara con núevos valores aleatorios
    */
    public static void generarMatriz(int[][] matriz){
        for(int i = 0; i < NUMERO_DE_EJERCICIOS; i++) {
            for(int j = 0; j < NUMERO_DE_EJERCICIOS; j++) {

                matriz[i][j] = RANDOM.nextInt(VALOR_MAX_MATRIZ) + VALOR_MIN_MATRIZ;
            }
        }
    }

    public static void main(String[] args) {
        // Bucle para generar los paneles uno al lado del otro sin superponerse e
        // inicializo el array de booleanos
        for (int numPanel = 0; numPanel < NUMERO_DE_PANELES; numPanel++) {
            PANELES[numPanel] = new Panel("Panel " + (numPanel + NORMALIZAR_NUMERO_DE_PANEL),
                    (numPanel * NORMALIZAR_POSICION_X_DE_PANEL),
                    NORMALIZAR_POSICION_Y_DE_PANEL);
            PANELES_LIBRES[numPanel] = true;
        }

        for(int numEquipo = 0; numEquipo < NUMERO_DE_EQUIPOS; numEquipo++) {
            EQUIPOS[numEquipo] = new Equipo(numEquipo);
            EQUIPOS[numEquipo].start();
        }


        for(int numEquipo = 0; numEquipo < NUMERO_DE_EQUIPOS; numEquipo++) {
            try {
                EQUIPOS[numEquipo].join();
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();;
            }
        }

        System.out.println("Fin del hilo principal."); //NOSONAR
    }

}
```

Clase Equipo

```
package ejercicio2; //NOSONAR

import java.util.Arrays;

@SuppressWarnings("java:S2696")
public class Equipo extends Thread{

        public static final int POSICION_DE_SPRINT = 0;
        public static final int PRIMER_EJERCICIO_SACAR_MAX = 1;

        private final int idEquipo;

        // Estructura con las 4 matrices
        private final int[][][] tablaDeMatricesDeEjercicios =
            new int[Main.NUMERO_DE_DEPORTISTAS_POR_EQUIPO]
            [Main.NUMERO_DE_EJERCICIOS][Main.NUMERO_DE_EJERCICIOS];
        // Matriz con las mejores marcas en cada deporte de cada deportista
        private final int[][] matrizResumenR = new int[Main.NUMERO_DE_DEPORTISTAS_POR_EQUIPO]
                [Main.NUMERO_DE_EJERCICIOS];

        public Equipo(int idEquipo) {
            this.idEquipo = idEquipo;
        }

        /**
        * Función para extraer las mejores marcas de una matriz de ejercicios
        * @param ejercicios matriz cuyas filas representan un ejercicio y contienen sus marcas.
        * @return devuelve un vector con las mejores marcas en cada ejercicio en orden
        */
        private static int[] mejoresMarcas(int[][] ejercicios) {
            int[] vectorDeMarcas = new int[Main.NUMERO_DE_EJERCICIOS];
            //Guardo sprint con el valor mínimo, ya que es una prueba de velocidad.
            if(Arrays.stream(ejercicios[POSICION_DE_SPRINT]).min().isPresent()) {
                vectorDeMarcas[POSICION_DE_SPRINT] =
                        Arrays.stream(ejercicios[POSICION_DE_SPRINT]).min().getAsInt();
            }
            // El resto de valores los guardo con el máximo salto, lanzamiento,
            // resistencia, y agilidad.
            for(int numEjercicios = PRIMER_EJERCICIO_SACAR_MAX;
                numEjercicios < Main.NUMERO_DE_EJERCICIOS; numEjercicios++) {
                if(Arrays.stream(ejercicios[numEjercicios]).max().isPresent()) {
                    vectorDeMarcas[numEjercicios] =
                            Arrays.stream(ejercicios[numEjercicios]).max().getAsInt();
                }
            }

            return vectorDeMarcas;
        }

        /**
        * Función que recibe una matriz de 4x5 y devuelve un string con el formato necesario
        * para imprimir.
        * @param matriz matriz que va a ser transformada en string (4 filas x 5 columnas)
        * @return matriz transformada en string con el formato que necesito
        */
        private static String matrizToString(int[][] matriz) {
            StringBuilder strMatriz = new StringBuilder();
            for (int i = 0; i < Main.NUMERO_DE_DEPORTISTAS_POR_EQUIPO; i++) {
                for (int j = 0; j < Main.NUMERO_DE_EJERCICIOS; j++) {
                    strMatriz.append(matriz[i][j]).append("␣");
                }
                strMatriz.append("\n");
            }
            return strMatriz.toString();
        }
```

```
        @Override
        public void run() {

            for(int secuenciasPorEquipo = 0; secuenciasPorEquipo < Main.NUMERO_DE_SECUENCIAS;
                    secuenciasPorEquipo++) {

                // Primero se genera una matriz por jugador
                for(int numJugadores = 0; numJugadores < Main.NUMERO_DE_DEPORTISTAS_POR_EQUIPO;
                        numJugadores++) {
                    Main.generarMatriz(tablaDeMatricesDeEjercicios[numJugadores]);
                }

                // Se seleccionan y guardan las mejores marcas de cada jugador
                // sprint, salto, lanzamiento, resistencia, y agilidad
                for(int numJugadores = 0; numJugadores < Main.NUMERO_DE_DEPORTISTAS_POR_EQUIPO;
                        numJugadores++) {
                    matrizResumenR[numJugadores] =
                        mejoresMarcas(tablaDeMatricesDeEjercicios[numJugadores]);
                }

                // Bloqueo el acceso a los paneles para que solo puedan estar el mismo
                // número de procesos que paneles hay.
                try {
                    Main.SEMAFORO_DE_PANELES.acquire();
                } catch (InterruptedException e) {
                    Thread.currentThread().interrupt();
                }
                // Bloqueo el acceso a la varible que puede ser usada por más de un proceso a la
                // vez para lectura y escritura, con un semáforo binario.
                try {
                    Main.MUTEX_VARIABLES_COMPARTIDAS.acquire();
                } catch (InterruptedException e) {
                    Thread.currentThread().interrupt();
                }

                // Llegado este punto hay un panel libre asegurado por lo que uso un while para
                // encontrar uno.
                int idPanel = 0;
                while(!Main.PANELES_LIBRES[idPanel]) {
                    idPanel++;
                }
                Main.PANELES_LIBRES[idPanel] = false;
                // Dejo libre el semáforo binario
                Main.MUTEX_VARIABLES_COMPARTIDAS.release();

                String mensajeParaPanel = ("Usando␣panel␣" + idPanel + "␣el␣hilo␣(equipo)␣"
                        + idEquipo + "\n\n" + "Matriz␣R␣(Resumen␣del␣equipo␣" + idEquipo
                        + ":␣mejores␣marcas)␣=\n" + matrizToString(matrizResumenR) + "\n"
                        + "Terminando␣de␣usar␣panel␣" + idPanel + "␣el␣hilo␣(equipo)␣"
                        + idEquipo + "\n");

                Main.PANELES[idPanel].escribir_mensaje(mensajeParaPanel);

                try {
                    Main.MUTEX_VARIABLES_COMPARTIDAS.acquire();
                } catch (InterruptedException e) {
                    Thread.currentThread().interrupt();
                }
                Main.PANELES_LIBRES[idPanel] = true;
                Main.MUTEX_VARIABLES_COMPARTIDAS.release();
                Main.SEMAFORO_DE_PANELES.release();
            }
        }

    }
```

3. Ejercicio 3: Monitores

3.1. Recursos no compartibles

No se podrán compartir, cada uno de los tornos y las colas para acceder estos. Por ello un monitor se encargará de gestionar el acceso a estos, el uso y su liberación.

En el caso de las zonas de entrenamiento, tenemos el mismo problema, por lo que se usará un monitor para gestionar el acceso a las zonas y su liberación. Además de esto también se usará el mismo monitor para gestionar el acceso a la bicicleta premium y su liberación, ya que es un recurso no compartible que usa una cola condition independiente.

Además, para poder acceder a la pantalla, se usará un monitor para evitar que entre más de un proceso a esta.

Uso tres monitores porque el código no se complica demasiado y cada uno de los monitores son independientes entre sí. Estos recursos no compartibles podrían estar gestionados de una infinidad de maneras, siempre que se cumpla la exclusión mutua en las secciones críticas.

3.2. Condiciones de sincronización

Para garantizar las condiciones de sincronización, se crean tres monitores: **MonitorTornos**, **MonitorZonas** y **MonitorPantalla**. El primero tiene un protocolo para el acceso y uso de los tornos y otro para su liberación. Evitando que varios procesos accedan simultáneamente a las variables de los tornos.

El segundo monitor tiene un protocolo para el acceso y uso de las zonas, otro para el uso de la bicicleta premium, otro para dejar libre lacola de la bicicleta premium y un último para la liberación de la zona tras su uso. Evitando que varios procesos accedan a las zonas simultáneamente o que varios lean y escriban a la vez donde no deberían.

Por último, el monitor de la pantalla se encarga de recibir lo que tiene que imprimir y usa esto en exclusión mutua, evitando que más de un proceso pueda acceder a la impresión por pantalla al mismo tiempo.

3.3. Pseudocódigo comentado

```
Algoritmo: Clase Main
Datos:
clientes : Cliente[]
monitorTornos : MonitorTornos
monitorZonasGimnasio : MonitorZonasGimnasio
monitorPantalla : MonitorPantalla
TIEMPO_MAX_DE_X_TORNO : Entero
TIEMPO_MAX_DE_Y_ZONA : Entero
// Inicialización de variables globales
NUMERO_DE_CLIENTES ← 50
TIEMPO_MAX_DE_X_TORNO ← 5
TIEMPO_MAX_DE_Y_ZONA ← 5
Función main()
    cobegin
        para i ← 1 a NUMERO_DE_CLIENTES hacer
            // Inicialización de un hilo cliente
            clientes[i].cliente(i);
        fin
    coend
fin
```

```
Algoritmo: Clase Cliente
Datos:
X : Entero
Y : Entero
usarBicicletaPremium : Booleano
// Inicializar variables del cliente
X ← random(1,TIEMPO_MAX_DE_X_TORNO)
Y ← random(1,TIEMPO_MAX_DE_Y_ZONA)
usarBicicletaPremium ← false
Función cliente(idCliente : Entero)
    idTorno ← monitorTornos.seleccionarYUsarTorno(X)
    sleep(X)
    monitorTornos.salirDeTorno(X, idTorno)
    idZonaYTiempos ← monitorZonasGimnasio.accederAZona(Y)
    si idZonaYTiempos[0] = CARDIO entonces
        usarBicicletaPremium ← random(1,10) ≤ 3
    fin
    // Crear mensaje con parámetros obtenidos previamente
    monitorPantalla.usarPantalla(mensaje)
    si usarBicicletaPremium entonces
        monitorZonasGimnasio.entrarEnBicicletaPremium()
    fin
    sleep(Y)
    si usarBicicletaPremium entonces
        monitorZonasGimnasio.salirDeBicicletaPremium()
    fin
    monitorZonasGimnasio.salirDeZona(Y, idZonaYTiempos[0])
fin
```

```
Algoritmo: MonitorTornos
Datos:
Privado numeroDeTornos : Entero
Privado tiempoEnTornos : Entero[]
Privado tornosLibres : Booleano[]
Privado colasDeTornos : Condition[]
// Inicializar variables del monitor
numeroDeTornos ← 3
para i ← 1 a numeroDeTornos hacer
    tiempoEnTornos[i] ← 0
    tornosLibres[i] ← true
fin
// Funciones del monitor
Función seleccionarYUsarTorno(X : Entero) : Entero
    // Buscar torno
    tiempoEnTornos [idTornoSeleccionado ] ← tiempoEnTornos
     [idTornoSeleccionado ] + X
    si tornosLibres[idTornoSeleccionado] = False entonces
        delay(colasDeTornos [idTornoSeleccionado ])
    fin
    tornosLibres[idTornoSeleccionado] ← False
    devolver idTornoSeleccionado
fin
Función salirDeTorno(X : Entero, idTorno : Entero)
    tornosLibres[idTorno] ← True
    tiempoEnTornos[idTorno] ← tiempoEnTornos[idTorno] − X
    resume(colasDeTornos [idTorno])
fin
```

```
Algoritmo: MonitorZonasGimnasio
Datos:
Privado numeroDeZonas : Entero
Privado maquinasPorZona : Entero
Privado maquinasPorZonaLibres : Entero[]
Privado colasDeZonas : Condition[]
Privado tiemposEnZonas : Entero[]
Privado bicicletaPremiumLibre : Booleano
Privado colaBicicletaPremium : Condition
// Inicializar variables del monitor
numeroDeZonas ← 4
maquinasPorZona ← 5
bicicletaPremiumLibre ← true
para i ← 1 a numeroDeZonas hacer
    maquinasPorZonaLibres[i] ← maquinasPorZona
    tiemposEnZonas[i] ← 0
fin
bicicletaPremiumLibre ← true
// Funciones del monitor
Función accederAZona(Y : Entero) : Entero[]
    // Buscar zona
    // Guarda en idZona
    // Guardo todos los tiempos
    idZonaYTiempos [0] ← idZona
    tiemposEnZonas [idZona] ← tiemposEnZonas [idZona] + Y
    si maquinasPorZonaLibres[idZona] ≤ 0 entonces
        delay(colasDeZonas [idZona])
    fin
    maquinasPorZonaLibres[idZona] ← maquinasPorZonaLibres[idZona] − 1
    devolver idZonaYTiempos
fin
```

```
Función entrarEnBicicletaPremium()
    si bicicletaPremiumLibre = False entonces
        delay(colaBicicletaPremium)
    fin
    bicicletaPremiumLibre ← False
fin
Función salirDeBicicletaPremium()
    bicicletaPremiumLibre ← True
    resume(colaBicicletaPremium)
fin
Función salirDeZona(Y : Entero, idZona : Entero)
    maquinasPorZonaLibres[idZona] ← maquinasPorZonaLibres[idZona] + 1
    tiemposEnZonas[idZona] ← tiemposEnZonas[idZona] − Y
    resume(colasDeZonas [idZona])
fin
```

```
Algoritmo: MonitorPantalla
Datos:
Private colaPantalla : Condition
Private pantallaLibre : Booleano
pantallaLibre ← true
Función usarPantalla(mensaje : Cadena)
    si pantallaLibre = False entonces
        delay(colaPantalla)
    fin
    pantallaLibre ← False
    // Imprimir mensaje en pantalla
    pantallaLibre ← True
    resume(colaPantalla)
fin
```

3.4. Cuestiones planteadas en los ejercicios

Según las condiciones de sincronización del problema,

a) **(0.5 puntos) Indica si la acción de entrenar es concurrente o en exclusión mutua y justifica.**
Es concurrente, ya que pueden haber varios procesos entrenando a la vez, siempre que hayan conseguido una máquina libre para ello. Como el entrenamiento se simula con un sleep, este segmento de código se puede realizar en paralelo junto con otros procesos que también estén entrenando, sin que esto suponga ningún problema.

b) **(0.75 puntos) ¿Qué tipo de monitor Java has usado? Justifica.**
He usado **condition**, ya que si uso synchronize no puedo usar más de una cola y no podría diferenciar entre procesos. Y como hay procesos que quieren acceder a diferentes zonas simuladas con colas es necesario usar condition. En el caso de la pantalla, como solo hay una y no necesito diferenciar entre procesos, podría usar synchronize, pero para mantener la consistencia en el código he decidido usar condition también para esta sección.

c) **(0.5 puntos) En el monitor, ¿has usado notify/signal o notifyAll/signalAll? Justifica**
Podría usar ambos, pero he decidido usar solo notify/signal, ya que no es necesario despertar a todos los procesos que esperan en una cola, porque al final solo uno de ellos va a poder ejecutarse y el resto se volverán a bloquear en la cola.

d) **(0.75 puntos) ¿Cómo se ha resuelto la exclusión mutua de la pantalla en este problema**
Está resuelto con el uso de un monitor que encapsula el uso de la pantalla en un procedimiento haciendo que solo un proceso pueda acceder a ella al mismo tiempo.

3.5. Código Java de los ejercicios comentado

Main

```
package ejercicio3; //NOSONAR

public class Main {

    public static final int NUMERO_DE_CLIENTES = 50;
    public static final int TIEMPO_MAX_DE_X_TORNO = 5;
    public static final int TIEMPO_MAX_DE_Y_ZONA = 5;
    public static final int TIEMPO_MIN = 1;

    protected static final Cliente[] CLIENTES = new Cliente[NUMERO_DE_CLIENTES];
    public static final MonitorTornos MONITOR_TORNOS = new MonitorTornos();
    public static final MonitorZonasGimnasio MONITOR_ZONAS_GIMNASIO =
        new MonitorZonasGimnasio();
    public static final MonitorPantalla MONITOR_PANTALLA = new MonitorPantalla();

    public static void main(String[] args) {
        for(int i = 0; i < NUMERO_DE_CLIENTES; i++) {
            CLIENTES[i] = new Cliente(i);
            CLIENTES[i].start();
        }
        for(Cliente cliente : CLIENTES) {
            try {
                cliente.join();
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }
        }

        System.out.println("Fin␣del␣hilo␣principal."); //NOSONAR
    }

}
```

Cliente

```
package ejercicio3; //NOSONAR

import java.util.Random;

public class Cliente extends Thread {

    private final int idCliente;
    private static final Random RANDOM = new Random();
    private static final int EMPEZAR_EN_UNO = 1;
    private int X = RANDOM.nextInt(Main.TIEMPO_MAX_DE_X_TORNO) + Main.TIEMPO_MIN;
    private int Y = RANDOM.nextInt(Main.TIEMPO_MAX_DE_Y_ZONA) + Main.TIEMPO_MIN;
    private boolean usarBicicletaPremium = false;

    public static final int POSICION_DE_ID_DE_ZONA = 0;
    public static final int POSICION_DE_CARDIO_EN_MONITOR = 0;
    public static final int CIEN_POR_CIENTO = 10;
    public static final int TREINTA_POR_CIENTO = 3;
    public static final int CARDIO = 1;
    public static final int FUERZA = 2;
    public static final int FUNCIONAL = 3;
    public static final int ESTIRAMIENTOS = 4;


    public Cliente(int idCliente) {
        this.idCliente = idCliente;
    }

    @Override
    public void run() {
        int idTorno = Main.MONITOR_TORNOS.seleccionarYUsarTorno(X);
        try {
            Thread.sleep(X);
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }
        Main.MONITOR_TORNOS.salirDeTorno(X, idTorno);
        int[] idZonaYTiempos = Main.MONITOR_ZONAS_GIMNASIO.accederAZonaYRecuperarTiempos(Y);
        int idZona = idZonaYTiempos[POSICION_DE_ID_DE_ZONA];

        // {0,1,2} es true, {3 ,4 ,5 ,6 ,7 ,8 , 9} es false
        if(idZona == POSICION_DE_CARDIO_EN_MONITOR) {
            usarBicicletaPremium = (RANDOM.nextInt(CIEN_POR_CIENTO) < TREINTA_POR_CIENTO);
        }

        String mensaje =
                "------------------------------------------------------------\n"
                + "Cliente " + this.idCliente + " ha pasado por el torno: " + idTorno
                + "\n"
                + "Tiempo en el torno (acceso): " + this.X + "\n"
                + "Zona elegida: " + (idZona+EMPEZAR_EN_UNO) + "\n"
                + "Tiempo de entrenamiento: " + this.Y + "\n"
                + "Estimación de espera (sin incluirse a sí mismo):\n"
                + "  Zona1(Cardio)=" + idZonaYTiempos[CARDIO]
                + ", Zona2(Fuerza)=" + idZonaYTiempos[FUERZA]
                + ", Zona3(Funcional)=" + idZonaYTiempos[FUNCIONAL] + "\n"
                + "Zona4(Estiramientos)=" + idZonaYTiempos[ESTIRAMIENTOS] + "\n"
                + "Espera bicicleta premium (si aplica)="
                + (usarBicicletaPremium ? "Sí" : "No") + "\n"
                + "------------------------------------------------------------\n";
        Main.MONITOR_PANTALLA.usarPantalla(mensaje);
        try {
            Thread.sleep(Y);
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }
        if(usarBicicletaPremium) {
            Main.MONITOR_ZONAS_GIMNASIO.salirDeBicicletaPremium();
        }
        Main.MONITOR_ZONAS_GIMNASIO.salirDeZona(Y, idZona);
    }
}
```

MonitorTornos

```java
package ejercicio3; //NOSONAR

import java.util.concurrent.locks.Condition;
import java.util.concurrent.locks.ReentrantLock;

public class MonitorTornos {
    private static final int NUMERO_DE_TORNOS = 3;
    private final int[] tiemposEnTornos = new int[NUMERO_DE_TORNOS];
    private final boolean[] tornosLibres = new boolean[NUMERO_DE_TORNOS];
    private final ReentrantLock l = new ReentrantLock();
    private final Condition[] colasDeTornos = new Condition[NUMERO_DE_TORNOS];

    private static final int POSICION_POR_DEFECTO = 0;

    public MonitorTornos() {
        for(int i = 0; i < NUMERO_DE_TORNOS; i++) {
            tiemposEnTornos[i] = 0;
            tornosLibres[i] = true;
            colasDeTornos[i] = l.newCondition();
        }
    }

    /**
    * Función que se encarga de seleccionar un torno libre y en caso de no haber uno libre
    * selecciona el primero con un mejor tiempo.
    * @return Devuelve el ID del torno elegido.
    */
    private int buscarTorno() {
        int idTornoSeleccionado = POSICION_POR_DEFECTO;
        for(int idTornoActual = 0; idTornoActual < NUMERO_DE_TORNOS; idTornoActual++) {
            if(tornosLibres[idTornoActual]) {
                return idTornoActual;
            }
            if(tiemposEnTornos[idTornoSeleccionado] > tiemposEnTornos[idTornoActual]) {
                idTornoSeleccionado = idTornoActual;
            }
        }
        return idTornoSeleccionado;
    }

    /**
    * Función que se encarga de seleccionar un torno para poder usarlo nada más salir de esta.
    * @param X es el tiempo que va a pasar en el torno quien lo pide
    * @return devuelve el ID del torno que se selecciona para usarlo
    */
    public int seleccionarYUsarTorno(int X) {
        l.lock();
        try {
            int idTornoSeleccionado = buscarTorno();
            tiemposEnTornos[idTornoSeleccionado] += X;
            while(!tornosLibres[idTornoSeleccionado]) {
                try {
                    colasDeTornos[idTornoSeleccionado].await();
                } catch (InterruptedException e) {
                    Thread.currentThread().interrupt();
                }
            }
            tornosLibres[idTornoSeleccionado] = false;
            return idTornoSeleccionado;
        } finally {
            l.unlock();
        }
    }

    /**
    * Función para liberar el torno que se ha usado.
    * @param X tiempo para restárselo al tiempo en la cola del torno.
    * @param idTorno se usa para saber que torno se va a liberar.
    */
    public void salirDeTorno(int X, int idTorno) {
        l.lock();
        try {
            tornosLibres[idTorno] = true;
            tiemposEnTornos[idTorno] -= X;
            colasDeTornos[idTorno].signal();
        } finally {
            l.unlock();
        }
    }

}
```

MonitorZonasGimnasio

```java
package ejercicio3; //NOSONAR

import java.util.ArrayList;
import java.util.Random;
import java.util.concurrent.locks.Condition;
import java.util.concurrent.locks.ReentrantLock;

public class MonitorZonasGimnasio {
    private static final int NUMERO_DE_ZONAS = 4;
    private static final int MAQUINAS_POR_ZONA = 5;

    private final int[] maquinasPorZonaLibres = new int[NUMERO_DE_ZONAS];
    private final Condition[] colasDeZonas = new Condition[NUMERO_DE_ZONAS];
    private final int[] tiemposEnZonas = new int[NUMERO_DE_ZONAS];
    private boolean bicicletaPremiumLibre = true;
    private final ReentrantLock l = new ReentrantLock();
    private final Condition colaBicicletaPremium = l.newCondition();

    // Constantes para que no hallan números mágicos
    private static final int MEJOR_ZONA_POR_DEFECTO = 0;
    private static final int TIEMPO_INICIAL_POR_ZONA = 0;
    private static final int ESPACIO_PARA_ZONA_Y_TIEMPOS = 1 + NUMERO_DE_ZONAS;
    private static final int POSICION_INICIAL_DE_TIEMPOS = 1;
    private static final Random RANDOM = new Random();
    private static final int POSICION_DE_ID_DE_ZONA = 0;

    public MonitorZonasGimnasio(){
        for (int i = 0; i < NUMERO_DE_ZONAS; i++) {
            maquinasPorZonaLibres[i] = MAQUINAS_POR_ZONA;
            colasDeZonas[i] = l.newCondition();
            tiemposEnZonas[i] = TIEMPO_INICIAL_POR_ZONA;
        }
    }

    /**
    * Función auxiliar para sacar una zono y todos los tiempos en las zonas en un array.
    * En caso de haber más de una zona libre se selecciona una aleatoria, si estuvieran
    * todas ocupadas se elige la que mejor tiempo tenga.
    * @return array que en la primera posición tiene el ID de la zona elegida
    * y en el resto de posiciones los tiempos de todas las zonas.
    */
    private int[] obtenerZonaYTodosLosTiempos(){

        int[] zonaElegidaYTiempos = new int[ESPACIO_PARA_ZONA_Y_TIEMPOS];
        ArrayList<Integer> zonasConMaquinasLibres = new ArrayList<>();
        int zonaLlenaConMejorTiempo = MEJOR_ZONA_POR_DEFECTO;
        for(int i = 0; i < NUMERO_DE_ZONAS; i++){
            if(maquinasPorZonaLibres[i] > 0){
                zonasConMaquinasLibres.add(i);
            }
            else if(tiemposEnZonas[zonaLlenaConMejorTiempo] > tiemposEnZonas[i]){
                zonaLlenaConMejorTiempo = i;
            }
            zonaElegidaYTiempos[i + POSICION_INICIAL_DE_TIEMPOS] = tiemposEnZonas[i];
        }
        if(!zonasConMaquinasLibres.isEmpty()){
            zonaElegidaYTiempos[POSICION_DE_ID_DE_ZONA] = zonasConMaquinasLibres
                    .get(RANDOM.nextInt(zonasConMaquinasLibres.size()));
        }
        else{
            zonaElegidaYTiempos[POSICION_DE_ID_DE_ZONA] = zonaLlenaConMejorTiempo;
        }
        return zonaElegidaYTiempos;
    }
```

```
    /**
    * Función que se encarga de seleccionar una zona y devolverla junto a los tiempos.
    * Si estuviera llena nos bloquearíamos hasta que se liberara una máquina.
    * @param Y es el tiempo que va a estar en la zona quien la pide.
    * @return posición de zona elegida en el primer elemento del array y el resto son
    * los tiempos.
    */
    public int[] accederAZonaYRecuperarTiempos(int Y) {
        l.lock();
        try {
            int[] idZonaYTiempos = obtenerZonaYTodosLosTiempos();
            int idZona = idZonaYTiempos[POSICION_DE_ID_DE_ZONA];
            tiemposEnZonas[idZona] += Y;

            while (maquinasPorZonaLibres[idZona] <= 0) {
                try {
                    colasDeZonas[idZona].await();
                } catch (InterruptedException e) {
                    Thread.currentThread().interrupt();
                }
            }

            maquinasPorZonaLibres[idZona]--;

            return idZonaYTiempos;
        } finally {
            l.unlock();
        }
    }

    public void entrarEnBicicletaPremium() {
        l.lock();
        try {
            while (!bicicletaPremiumLibre) {
                try {
                    colaBicicletaPremium.await();
                } catch (InterruptedException e) {
                    Thread.currentThread().interrupt();
                }
            }
            bicicletaPremiumLibre = false;
        } finally {
            l.unlock();
        }
    }

    public void salirDeBicicletaPremium() {
        l.lock();
        try {
            bicicletaPremiumLibre = true;
            colaBicicletaPremium.signal();
        } finally {
            l.unlock();
        }
    }

    public void salirDeZona(int Y, int idZona) {
        l.lock();
        try {
            maquinasPorZonaLibres[idZona]++;
            tiemposEnZonas[idZona] -= Y;
            colasDeZonas[idZona].signal();
        } finally {
            l.unlock();
        }
    }
}
```

MonitorPantalla

```java
package ejercicio3; //NOSONAR

import java.util.concurrent.locks.Condition;
import java.util.concurrent.locks.ReentrantLock;

@SuppressWarnings("java:S106")
public class MonitorPantalla {
    private final ReentrantLock l = new ReentrantLock();
    private final Condition colaPantalla = l.newCondition();
    private boolean pantallaLibre = true;

    /**
    * Recibe una cadena de texto y la imprime en exclusión mutua.
    * @param mensaje es la cadena para imprimir.
    */
    public void usarPantalla(String mensaje) {
        l.lock();
        try {
            while (!pantallaLibre) {
                try {
                    colaPantalla.await();
                } catch (InterruptedException e) {
                    Thread.currentThread().interrupt();
                }
            }
            pantallaLibre = false;
            System.out.println(mensaje);
            pantallaLibre = true;
            colaPantalla.signal();
        } finally {
            l.unlock();
        }
    }
}
```

4. Ejercicio 4: Paso de Mensajes Asíncrono

Tengo dos puentos que se pueden considerar mejoras: En este ejercicio he incluido una mejora para que el hilo principal termine al no ser un bucle infinito el controlador de acceso. En el caso de pasar varios parametros por un buzón, he usado un array de enteros ya que este es serializable y se puede pasar a través de un buzón para luego hazer el casting de Object a int[].

4.1. Recursos no compartibles

Los recursos no compartibles son las colas, tanto la rápida como la lenta y la pantalla.

4.2. Condiciones de sincronización

Las condiciones de sincronización se garantizan gracias al uso de un controlador que simula el paso de mensajes asíncrono, este tiene condiciones de guarda en los casos del uso de los recursos no compartibles. De esta forma, se garantiza la exclusión mutua mientras un proceso use uno de estos recursos.

En el caso de la asignación de un tiempo no es necesarío una condición de guarda porque no se esta protegiendo ningún recurso. Para liberar una cola y liberar la pantalla, no es necesario aplicar condiciones de guarda, ya que solo hará una de estas acciones un proceso que previamente halla accedido a uno de estos recursos.

Dicho todo esto solo es necesario aplicar condiciones de guarda para el uso de colas y el uso de pantalla.

Además al ser paso de mensajes asíncrono, en algunos casos, es necesario sincronizar al **controlador** con el **aficionado**, y al no haber **sends** bloqueantes se simulan con el uso de **receive** justo después del send, haciendo así que el proceso espere a la contestación del controlador. El uso de este mecanismo solo es necesario al pedir un tiempo, al pedir una cola y al pedir la pantalla. Para su liberación no es necesario, ya que el aficionado no necesita esperar a que el controlador le confirme que ha liberado el recurso.

4.3. Pseudocódigo comentado

```
Algoritmo: Clase Main
Datos:
NUMERO_DE_AFICIONADOS : Entero
controladorDeAcceso : ControladorDeAcceso
aficionados : Aficionado[]
buzonesAficionados : Buzon[]
determinarColaConTiempo : Buzon
pedirColaRapida : Buzon
pedirColaLenta : Buzon
liberarCola : Buzon
pedirPantalla : Buzon
TOKEN_PANTALLA : Caracter
COLA_RAPIDA : Caracter
COLA_LENTA : Caracter
// Inicialización de variables globales
NUMERO_DE_AFICIONADOS ← 50
COLA_RAPIDA ← R
COLA_LENTA ← L
TOKEN_PANTALLA ← P
Función main()
    send(pedirPantalla, TOKEN_PANTALLA)
    cobegin
        controladorDeAcceso.iniciar()
        para i ← 1 a NUMERO_DE_AFICIONADOS hacer
            // Inicialización de un hilo aficionado y
            // ejecución.
            aficionados[i].aficionado(i, buzonesAficionados [i]);
        fin
    coend
fin
```

```
Algoritmo: Clase Aficionado
Datos:
NUMERO_DE_ITERACIONES : Entero
idAficionado : Entero
caminarABanos : Entero
T : Entero
buzonAficionado : Buzon
NUMERO_DE_ITERACIONES ← 5
Función aficionado(idAficionado : Entero, buzonAficionado : Buzon)
    para i ← 1 a NUMERO_DE_ITERACIONES hacer
        sleep(random(1, caminarABanos))
        T ← random(1, 10)
        send(determinarColaConTiempo, {idAficionado, T})
        receive(BuzonAficionado, colaSeleccionada)
        si colaSeleccionada = COLA_RAPIDA entonces
            send(pedirColaRapida, idAficionado)
        en otro caso
            send(pedirColaLenta, idAficionado)
        fin
        receive(buzonAficionado, null)
        sleep(T)
        send(liberarCola, colaSeleccionada)
        receive(pedirPantalla, TOKEN_PANTALLA)
        // Imprimir en pantalla
        send(pedirPantalla, TOKEN_PANTALLA)
    fin
fin
```

```
Algoritmo: Clase ControladorDeAcceso
Datos:
libreColaRapida : Booleano
libreColaLenta : Booleano
// Inicializar booleanos de guardas
libreColaRapida ← true
libreColaLenta ← true
Función iniciar()
   while(true)
      select
           // Selector para pedir una cola.
           receive(determinarColaConTiempo, {idAficionado, T})
           si T ≤ 5 entonces
            send(buzonesAficionados [idAficionado], COLA_RAPIDA)
           en otro caso
            send(buzonesAficionados [idAficionado], COLA_LENTA)
         or  // Selector de cola rápida con guarda
           when libreColaRapida = true =>
           receive(pedirColaRapida, idAficionado)
           libreColaRapida ← false
           send(buzonesAficionados [idAficionado], null)
         or  // Selector de cola lenta con guarda
           when libreColaLenta = true =>
           receive(pedirColaLenta, idAficionado)
           libreColaLenta ← false
           send(buzonesAficionados [idAficionado], null)
         or   // Selector para salir de cola rápida
           receive(liberarCola, colaSeleccionada)
           si colaSeleccionada = COLA_RAPIDA entonces
            libreColaRapida ← true
           en otro caso
            libreColaLenta ← true
fin
```

4.4. Cuestiones planteadas en los ejercicios

a) **(0.5 puntos sobre 10) ¿Se pueden usar simultáneamente los dos tornos? Justifica la respuesta**
Si, ya que el controlador solo se encarga de dar acceso a uno de los dos si esta libre, por lo que puede darle acceso a dos procesos uno con el rápido y otro con el lento, siempre que ambos estén libres. Por lo que uno puede estar usando el lento hacer un **sleep** y mientras estar otro en el rápido haciendo lo mismo.

b) **(0.5 puntos sobre 10) ¿Cómo has resuelto la exclusión mutua de la pantalla?**
Para resolver esto he usado un buzón que contiene un **TOKEN_PANTALLA** al inicio del código, este actúa como un semaforo binario otorgando acceso a la pantalla a un solo proceso a la vez. El proceso que use el **receive** para pedir el token debe esperar a que este este en el buzón para poder cogerlo y asi usar la pantalla el solo hasta que lo deje de nuevo en el buzón cuando termine de usar la pantalla.

4.5. Código Java de los ejercicios comentado

Clase Main

```
package ejercicio4; //NOSONAR

import messagepassing.MailBox;

public class Main {
    public static final int NUMERO_DE_AFICIONADOS = 50;
    protected static final Aficionado[] AFICIONADOS = new Aficionado[NUMERO_DE_AFICIONADOS];
    protected static final MailBox[] buzonesAficionados = new MailBox[NUMERO_DE_AFICIONADOS];
    public static final MailBox determinarColaConTiempo = new MailBox();
    public static final MailBox pedirColaRapida = new MailBox();
    public static final MailBox pedirColaLenta = new MailBox();
    public static final MailBox liberarCola = new MailBox();
    public static final MailBox pedirPantalla = new MailBox();

    public static final ControladorDeAcceso CONTROLADOR_DE_ACCESO =
            new ControladorDeAcceso(determinarColaConTiempo, pedirColaRapida, pedirColaLenta,
            liberarCola, buzonesAficionados);

    public static final char COLA_RAPIDA = 'R';
    public static final char COLA_LENTA = 'L';
    public static final char TOKEN_PANTALLA = 'P';

    public static void main(String[] args) {
        pedirPantalla.send(TOKEN_PANTALLA);
        CONTROLADOR_DE_ACCESO.start();
        for (int i = 0; i < NUMERO_DE_AFICIONADOS; i++) {
            buzonesAficionados[i] = new MailBox();
            AFICIONADOS[i] = new Aficionado(i, buzonesAficionados[i], determinarColaConTiempo,
            pedirColaRapida, pedirColaLenta, liberarCola, pedirPantalla);
            AFICIONADOS[i].start();
        }
        for (int i = 0; i < NUMERO_DE_AFICIONADOS; i++) {
            try {
                AFICIONADOS[i].join();
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }
        }

        System.out.println("Fin del hilo principal."); //NOSONAR
    }
}
```

Clase Aficionado

```
package ejercicio4; //NOSONAR

import messagepassing.MailBox;

import java.util.Random;

public class Aficionado extends Thread {
    public static final int NUMERO_ITERACIONES = 5;
    public static final int CAMINAR_A_BANOS = 5;
    public static final int TIEMPO_MINIMO = 1;
    public static final int TIEMPO_MAXIMO_TORNO = 10;
    private static final Random RANDOM = new Random();

    public static final int PETICIONES_POR_HILO_A_CONTROLADOR = 3;

    private final int idAficionado;
    private final MailBox buzonAficionado;
    private final MailBox determinarColaConTiempo;
    private final MailBox pedirColaRapida;
    private final MailBox pedirColaLenta;
    private final MailBox liberarCola;
    private final MailBox pedirPantalla;

    public Aficionado(int idAficionado,
                      MailBox buzonAficionado,
                      MailBox determinarColaConTiempo,
                      MailBox pedirColaRapida,
                      MailBox pedirColaLenta,
                      MailBox liberarCola,
                      MailBox pedirPantalla) {
        this.idAficionado = idAficionado;
        this.buzonAficionado = buzonAficionado;
        this.determinarColaConTiempo = determinarColaConTiempo;
        this.pedirColaRapida = pedirColaRapida;
        this.pedirColaLenta = pedirColaLenta;
        this.liberarCola = liberarCola;
        this.pedirPantalla = pedirPantalla;
    }

    @Override
    public void run() {
        int T = 0;
        int tiempoParaIrABano = 0;
        for(int i = 0; i < NUMERO_ITERACIONES; i++) {
            tiempoParaIrABano = RANDOM.nextInt(CAMINAR_A_BANOS) + TIEMPO_MINIMO;
            try {
                Thread.sleep(tiempoParaIrABano);
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }
            T = RANDOM.nextInt(TIEMPO_MAXIMO_TORNO) + TIEMPO_MINIMO;
            int[] tuplaIDTiempo = new int[]{idAficionado, T};
            determinarColaConTiempo.send(tuplaIDTiempo);
            char colaSeleccionada = (char) buzonAficionado.receive();

            if (colaSeleccionada == Main.COLA_RAPIDA) {
                pedirColaRapida.send(idAficionado);
            } else {
                pedirColaLenta.send(idAficionado);
            }

            buzonAficionado.receive();
            try{
                Thread.sleep(T);
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }
            liberarCola.send(colaSeleccionada);
            pedirPantalla.receive();
            System.out.println("Aficionado␣" + idAficionado + "␣ha␣usado␣la␣cola␣"
            + colaSeleccionada + "\n"
            + "Tiempo␣de␣validación␣=␣" + T + "\n"
            + "Thread.sleep(" + T + ")\n"
            + "Aficionado␣" + idAficionado + "␣liberando␣la␣cola␣" + colaSeleccionada "\n");
            pedirPantalla.send(Main.TOKEN_PANTALLA);
        }
    }

}
```

Clase ControladorDeAcceso

```
package ejercicio4; //NOSONAR

import messagepassing.MailBox;
import messagepassing.Selector;

public class ControladorDeAcceso extends Thread {
    private static final int TIEMPO_MAXIMO_COLA_RAPIDA = 5;
    private static final int POSICION_ID_CLIENTE = 0;
    private static final int POSICION_TIEMPO_CLIENTE = 1;
    private static final String CADENA_VACIA = "";
    private static final int ITERACIONES_DEL_CONTROLADOR = Main.NUMERO_DE_AFICIONADOS *
        Aficionado.NUMERO_ITERACIONES * Aficionado.PETICIONES_POR_HILO_A_CONTROLADOR;
    private boolean libreColaRapida = true;
    private boolean libreColaLenta = true;

    private final Selector selector = new Selector();

    private final MailBox determinarColaConTiempo;
    private final MailBox pedirColaRapida;
    private final MailBox pedirColaLenta;
    private final MailBox liberarCola;
    private final MailBox[] buzonesAficionados;

    public ControladorDeAcceso(MailBox determinarColaConTiempo,
                           MailBox pedirColaRapida,
                           MailBox pedirColaLenta,
                           MailBox liberarCola,
                           MailBox[]  buzonesAficionados) {
        this.determinarColaConTiempo = determinarColaConTiempo;
        this.pedirColaRapida = pedirColaRapida;
        this.pedirColaLenta = pedirColaLenta;
        this.liberarCola = liberarCola;
        this.buzonesAficionados = buzonesAficionados;

        selector.addSelectable(determinarColaConTiempo, false);
        selector.addSelectable(pedirColaRapida, false);
        selector.addSelectable(pedirColaLenta, false);
        selector.addSelectable(liberarCola, false);

        pedirColaRapida.setGuardValue(libreColaRapida);
        pedirColaLenta.setGuardValue(libreColaLenta);
    }

    @Override
    public void run() {
        for(int i = 0; i < ITERACIONES_DEL_CONTROLADOR; i++) {
            switch (selector.selectOrBlock()){
                case 1:{ // Selector para pedir cola
                    int[] tuplaIDTiempo = (int[]) determinarColaConTiempo.receive();
                    int id = tuplaIDTiempo[POSICION_ID_CLIENTE];
                    int tiempo = tuplaIDTiempo[POSICION_TIEMPO_CLIENTE];
                    if(tiempo <= TIEMPO_MAXIMO_COLA_RAPIDA)
                        buzonesAficionados[id].send(Main.COLA_RAPIDA);
                    else
                        buzonesAficionados[id].send(Main.COLA_LENTA);
                    break;
                }
                case 2: { // Pedir cola rápida
                    int id = (int) pedirColaRapida.receive();
                    libreColaRapida = false;
                    buzonesAficionados[id].send(CADENA_VACIA);
                    break;
                }
                case 3:{ // Pedir cola lenta
                    int id = (int) pedirColaLenta.receive();
                    libreColaLenta = false;
                    buzonesAficionados[id].send(CADENA_VACIA);
                    break;
                }
                case 4: { // Liberar cola
                    char colaSeleccionada = (char) liberarCola.receive();
                    if (colaSeleccionada == Main.COLA_RAPIDA) {
                        libreColaRapida = true;
                    } else {
                        libreColaLenta = true;
                    }
                }
            }
        }
    }
}
```